

Project 2 Report

Project Overview/Goals:

The goal of this project was to be able to generate next text by creating a “Random Writer” which learns which characters follow in multiple short sequences in a source text. The program records the occurrences and uses them to choose the next character randomly. There’s a sequence of k characters which we call the ‘seed’, and ‘ k ’ is the analysis level, which is how far you are looking back in the sequence.

For example, at level 2, take the seed “th,” and we can expect this to be followed by the letter ‘e’ more often than by any other letter. This assumption we make is actually a form of “Random Writer” that our minds automatically do: we’ve seen the letter ‘e’ follow the sequence “th” so many times, we expect to see an e. This project’s goal was to create a version of this.

The main goal of this project was to be able to calculate the probabilities of the following characters in a source text, while also being able to write the correct characters based off of the level.

Scope/Quality/Limitations

The implementation reads and writes plain text files, uses only non-negative analysis levels and non-negative output lengths. The source must be longer than the level. The implementation handles spaces, punctuation, capitalization, and line breaks as given in the model.

Zero output length results in a blank output file being created or overwritten, but no characters written to it.

Only ordinary text is suitable for this, and what this means is that emojis may not be able to fit in a standard Java char value and it would have to be split.

Assumptions

- The input is plain text that can be interpreted in standard run time
- Source text fits into the available memory
- The source text ends normally: what this means is that the last character doesn't wrap around to the beginning through any form of circular indexing
- Length counts the Java chars including the whitespace instead of words
- readText runs before and succeeds before any writing is done
- There are different random outputs for runs and that is expected: I didn't base the tests off of a single passage, more so if the rules were followed or not.

Algorithm and Implementation

Main(String[] args)

The main method takes the four required arguments: a source filename, a result filename, a level, and an output length. It checks the argument count, converts the last two strings to integers (where non-integer or negative values cause an error on System.err and an early return), creates a processor, reads the source, and writes the output (with separate error messages for read errors and source too short, and error messages when opening or writing the output file).

Create Processor(int level) and the constructor

A new Random Writer is returned as a Text Processor; and allows tests to create an object despite the private constructor.

The constructor rejects negative levels, stores the level, and creates a single Random object which is used for each choice.

Read Text(String inputFilename)

The method clears any text that exists before, reads characters into a String Builder through a Buffered Reader, then converts to a string, checking that character count > level

Then it creates `nextText`, which maps seed strings to List of following characters: The source text itself is stored for later seed choice.

Duplicates are kept in the lists; for example `[a, a, b]` yields a $2/3$ probability for a versus $1/3$ for b if `random.nextInt` chooses index 0 or 1. Percentages don't need to be calculated when generating, because the stored duplicates give "weight" for choosing an index, so one is more likely to be chosen than another.

`Write Text(String outputFilename, int length)`

The method doesn't write before a successful read (and refuses negative length).

Opening a Buffered Writer, it picks an initial seed; for each output character, it looks at the seed's followers; if empty, it chooses another seed until one is nonempty; then it picks one index randomly using `random.nextInt` in its range; print the character; and movies the seed forward using `advance(seed, char)`. An output character is generated each time.

`Random Seed()` and `advance(String seed, char c)`

`randomSeed` returns the empty string at level 0; if not that, it chooses a source position with `random.nextInt(sourceText.length() - level + 1)`, gets a string from it, and adds one to its index to include the last complete seed position.

`Advance` takes off the first character of seed and adds the new one to the end; for example, "th" followed by e becomes "he." At level 0, seed is always empty.

`Get Followers(String seed)`

The test method returns the List stored for a given seed as `nextText.get(seed)`, or null if seed is absent; and this was used for white box texting

Testing

1. Test Create Processor: Black-Box

- The test involves a level 2 processor, confirms it is not null, and verifies that it is a RandomWriter.
- It then has a level -1 processor, where an IllegalArgumentException is expected.
- The failure happens only after creation to confirm that the invalid level fails as intended.
- The catch block only accepts the expected exception.

2. Test Read Text: Black-Box

- The level 2 processor from before reads each book in sequence, with successful completion expected.
- The provided books are reasonably representative texts containing line breaks, spaces, and punctuation.
- Next, an empty source is passed in. readText has to reject it with an IllegalArgumentException because its length is not greater than 2.
- This check ensures that the books can be read;

4. Test Follower Counts: White-Box

- Reads the source string "aaaabaa" at level 2.
- getFollowers should return:

- ['a', 'a', 'b'] for the seed "aa"

-
- ['a'] for the seed "ab"
-
- ['a'] for the seed "ba"
-
- null for the missing seed "zz"
- This shows the $\frac{1}{3}$ vs $\frac{2}{3}$ probability for b and a respectively that I previously mentioned

5. Test Level Zero Followers and Additional Edge Cases

- At level 0, a source text that has two 'a' characters, a space, a 'b', an exclamation point, and a new line should appear unchanged in the empty seed's follower list.
- This verifies the level 0 model, ensuring whitespace and punctuation are preserved correctly because nothing is looked back at.
- Then there was the testNegativeLength method which takes a valid book and requests a length of -1, and from that we can expect an IllegalArgumentException

Interesting Results

The interesting result I found was that duplicate entries actually gave the random choosing an innate sense of weighted probability. I mention this in my read me as well because I talked to Vish and Sri ram about this and found that there was a $\frac{2}{3}$ vs $\frac{1}{3}$ probability split for the example I showed in my test case which made 'a' more likely to be selected, so in the case of reading an entire book, 'e' would appear so many more times that the program would 'randomly' choose e after seeing the sequence 'th'

Problems Encountered

I actually was missing a closing brace in the test class and it took me a solid 2 hours to find it because I didn't know where I messed up, but after that I was able

to figure it out. Also I had to check errors without accidentally passing a test because if I caught an IOException and printed a message, I had a couple broken tests which did that and I thought they were successful but they really weren't. So I had to figure out a way to deal with that as well.

Social Impact

Random Writer shows us how a program can imitate patterns based off of data sets we feed it without really understanding them. I feel like this is the genuine difference between Man and AI because the understanding of the material is what sets us apart. This experiment was nice to show how probability controls speech patterns but the output shouldn't really be taken for anything more than a simple project. What I mean by that is that more advanced text generators can help us fix our writing but they can't really understand the sentiment of our writing, so they can sometimes ruin the text more than help us revise it. So I feel like because of it, the social impact of this program is to show us how AI is really different from society as a whole, and how we can't totally depend on it for communication.